

Համացանցային արձանագրություններ, Proxy և վեբ մշակման միջավայր

1. Վեբ հարցման ցանցային շերտերը

Վեբ էջ բացելիս բրաուզերը միանգամից «էջ» չի ստանում: Այն ձևավորում է HTTP հարցում, տվյալները բաժանվում են ցանցային շերտերի պահանջներին համապատասխան և փոխանցվում են ցանցով: Ստացող կողմում կատարվում է հակառակ գործընթացը՝ տվյալները հասնում են վեբ սերվերին, սերվերը մշակում է հարցումը և կազմում HTTP պատասխան: Այս շերտավորումը թույլ է տալիս յուրաքանչյուր արձանագրության լուծել սահմանափակ և հստակ խնդիր:

Կիրառական շերտում HTTP-ը սահմանում է հարցման իմաստը՝ ինչ ռեսուրս է պահանջվում և ինչ գործողություն պետք է կատարվի: Տրանսպորտային շերտում TCP-ը կամ QUIC-ը կազմակերպում են տվյալների փոխանցումը երկու համակարգերի միջև: Համացանցային շերտում IP-ը պատասխանատու է փաթեթների հասցեավորման և երթուղավորման համար: Ֆիզիկական ու կապուղային շերտերը տվյալները իրականում տեղափոխում են Wi-Fi-ի, Ethernet-ի կամ այլ միջավայրի միջոցով:

Օրինակ՝ բրաուզերում որևէ էջ բացելիս HTTP շերտը կարող է ստեղծել GET հարցում, TCP/QUIC շերտը՝ այն փոխանցման համար բաժանել հոսքերի կամ փաթեթների, իսկ IP շերտը՝ ուղղորդել դեպի նպատակային համակարգ: Եթե էջը չի բացվում, շերտերի գաղափարը օգնում է հասկանալ՝ խնդիրը բրաուզերի հարցման, ապահով կապի, ցանցային միացման, թե սերվերի մեջ է: Այս տրամաբանությամբ են աշխատում նաև ախտորոշման գործիքները:

Շերտ	Հիմնական օրինակ	Ի՞նչ է որոշում
Կիրառական	HTTP, WebSocket	Հարցման իմաստը և տվյալների ձևաչափը
Տրանսպորտային	TCP, QUIC	Կապը, հոսքերը և փոխանցման հուսալիությունը
Համացանցային	IP	Փաթեթների հասցեավորումը և երթուղին
Կապուղային	Ethernet, Wi-Fi	Տվյալների տեղային փոխանցումը

2. Պորտ և endpoint

Միևնույն համակարգի վրա կարող են միաժամանակ աշխատել վեբ սերվեր, տվյալների բազա, փոստային ծառայություն և այլ ծրագրեր: Միայն համակարգի հասցեն բավարար չէ հասկանալու համար, թե տվյալը դրանցից որին պետք է հասնի: Պորտը 0-ից 65535 միջակայքի թվային նույնացուցիչ է, որով օպերացիոն

համակարգը տարբերակում է ցանցային ծառայությունները: HTTP-ի համար սովորաբար կիրառվում է 80 պորտը, իսկ HTTPS-ի համար՝ 443-ը:

IP հասցեի կամ host-ի և պորտի համադրությունը հաճախ անվանում են endpoint ցանցային իմաստով: Ծրագրային API-ների դեպքում endpoint բառը կարող է նշանակել նաև կոնկրետ URL և գործողություն, օրինակ՝ GET /api/products: Այս երկու իմաստները կապված են, բայց նույնը չեն. առաջինը ցույց է տալիս ցանցային ծառայության մուտքի կետը, երկրորդը՝ ծառայության տրամադրած կոնկրետ գործողությունը: Ուսանողը պետք է տարբերակի այս կիրառությունները ըստ համատեքստի:

Օրինակ՝ http://localhost:3000 հասցեն նշանակում է, որ բրաուզերը կապվում է նույն համակարգչում աշխատող 3000 պորտի ծառայությանը: Եթե ծրագիրը իրականում աշխատում է 5173 պորտում, 3000 հասցեն բացելիս կարելի է ստանալ connection refused սխալ: Պորտի համարը URL-ում կարելի է չգրել, երբ օգտագործվում է տվյալ scheme-ի ստանդարտ պորտը: Այդ պատճառով https://example.com հասցեում 443-ը սովորաբար տեսանելի չէ:

```
http://localhost:3000
https://example.com:443
GET /api/products
```

3. Proxy միջնորդ սերվեր

Proxy-ն ծրագրային կամ սարքային միջնորդ է, որը հարցումը ստանում է մի կողմից և այն փոխանցում մյուս կողմին: Հաճախորդը կամ սերվերը կարող է չիմանալ միջնորդի ներքին աշխատանքը, բայց ցանցային կապը փաստացի անցնում է proxy-ի միջոցով: Այն կարող է փոխել կամ ավելացնել վերնագրեր, գրանցել հարցումները, կիրառել հասանելիության կանոններ և օգտագործել քեշ: Proxy-ն ինքնին նպատակ չէ, այլ ենթակառուցվածքային գործիք՝ վերահսկման, անվտանգության և արագության համար:

Proxy-ի կարևոր առավելությունն այն է, որ ընդհանուր քաղաքականությունը կիրառվում է մեկ կենտրոնական կետում: Օրինակ՝ կազմակերպությունը կարող է մեկ proxy-ով սահմանափակել անցանկալի կայքերը, իսկ վեբ ծառայությունը՝ մեկ reverse proxy-ով ստուգել HTTPS կապը բոլոր ներքին սերվերների համար: Միջնորդը նաև կարող է թաքցնել ներքին ցանցի կառուցվածքը և նվազեցնել անմիջական հասանելիությունը վերջնական համակարգերին: Սակայն սխալ կարգավորումը կարող է դառնալ ուշացման, գաղտնիության կամ անվտանգության խնդիր:

Հարցման ուղու մեջ proxy-ի առկայությունը կարելի է նկատել որոշ HTTP վերնագրերից, սերվերի արձագանքման ժամանակից կամ ցանցային կարգավորումներից: Օրինակ՝ Via, Forwarded և X-Forwarded-For վերնագրերը կարող են փոխանցել միջնորդների մասին տեղեկություն, թեև դրանց առկայությունը միշտ պարտադիր չէ: Պետք է հիշել, որ այդ վերնագրերին անվերապահ վստահել չի կարելի, եթե դրանք չեն ավելացվում վստահելի reverse proxy-ի կողմից: Անվտանգ համակարգերում վստահության սահմանը հստակ կարգավորվում է:

4. Forward Proxy

Forward Proxy-ն աշխատում է հաճախորդի անունից: Բրաուզերը կամ օպերացիոն համակարգը հարցումը ուղարկում է proxy-ին, իսկ proxy-ն այն փոխանցում է

արտաքին կայքին: Արտաքին սերվերը կապը տեսնում է proxy-ի հասցեից և միշտ չէ, որ անմիջապես տեսնում է ներքին հաճախորդի հասցեն: Այս մոտեցումը սովորաբար կիրառվում է կազմակերպության ներսից դեպի համացանց գնացող կապը վերահսկելու համար:

Կազմակերպությունը կարող է forward proxy-ով արգելել որոշ կատեգորիայի կայքեր, պահանջել օգտատիրոջ նույնականացում և գրանցել այցելությունների պատմությունը: Այն կարող է նաև քեշավորել հաճախ ներբեռնվող նյութերը՝ նվազեցնելով արտաքին տրաֆիկը: HTTPS կայքերի դեպքում սովորաբար օգտագործվում է CONNECT մեթոդով tunnel, որի միջոցով ստեղծվում է կողավորված կապ նպատակային սերվերի հետ: Եթե կազմակերպությունը վերծանում է HTTPS տրաֆիկը, դրա համար անհրաժեշտ են հատուկ հավաստագրեր և հստակ անվտանգության քաղաքականություն:

Օրինակ՝ համալսարանի համակարգչային լսարանի բոլոր համակարգիչները կարող են համացանց դուրս գալ մեկ forward proxy-ով: Եթե որևէ կայք արգելափակված է, proxy-ն կարող է վերադարձնել սեփական 403 պատասխանը՝ նույնիսկ առանց վերջնական կայքին դիմելու: Այս դեպքում արգելքը գալիս է ոչ թե կայքից, այլ կազմակերպության միջնորդ համակարգից: Network բաժնի և proxy կարգավորումների համատեղ ստուգումը օգնում է տարբերել այդ դեպքերը:

5. Reverse Proxy

Reverse Proxy-ն աշխատում է սերվերային ենթակառուցվածքի անունից: Օգտատերը կապվում է մեկ հրապարակային հասցեի հետ, իսկ reverse proxy-ն որոշում է, թե ներքին որ սերվերին փոխանցի հարցումը: Այդ պատճառով մի քանի application server-ներ կարող են արտաքին աշխարհին երևալ որպես մեկ ծառայություն: Ներքին սերվերները հաճախ հասանելի չեն անմիջապես համացանցից:

Reverse proxy-ն կարող է կատարել TLS termination, այսինքն՝ ընդունել ու վերծանել HTTPS կապը, այնուհետև հարցումը փոխանցել ներքին ծառայությանը: Այն կարող է նաև հավասարակշռել ծանրաբեռնվածությունը, ստուգել օգտատիրոջ մուտքի իրավունքը, սահմանափակել հարցումների հաճախականությունը և քեշավորել պատասխանները: Եթե մեկ ներքին սերվեր չի աշխատում, proxy-ն կարող է հարցումը ուղարկել մյուս առողջ սերվերին: Այս հնարավորությունները բարձրացնում են ծառայության հասանելիությունն ու կառավարելիությունը:

Օրինակ՝ shop.example.com հասցեին եկող /api հարցումները կարող են փոխանցվել backend սերվերին, իսկ /images հարցումները՝ ստատիկ ֆայլերի սերվերին: Օգտատերը շարունակում է տեսնել նույն դոմենը և չի իմանում, թե ներսում քանի ծառայություն է աշխատում: Nginx-ը reverse proxy-ի տարածված օրինակ է, բայց նույն դերը կարող են կատարել նաև cloud load balancer-ները և այլ ծրագրեր: Սխալ 502 Bad Gateway պատասխանը հաճախ ցույց է տալիս, որ reverse proxy-ն չի կարողացել ճիշտ պատասխան ստանալ ներքին սերվերից:

```
Օգտատեր → Reverse Proxy → Web Server 1
                                → Web Server 2
                                → API Server
```

6. URI և URL

URI-ն ընդհանուր հասկացություն է, որը նույնականացնում է ռեսուրսը: URL-ը URI-ի տեսակ է, որը նկարագրում է նաև ռեսուրսին հասնելու եղանակն ու տեղը: Գործնական վեբ ծրագրավորման մեջ ամենից հաճախ աշխատում ենք URL-ների հետ: URL-ի մասերը հասկանալի կարևոր է հղումներ կազմելու, API կանչեր կատարելու և Network բաժնի տվյալները կարդալու համար:

URL-ը կարող է ունենալ scheme, host, port, path, query և fragment: Scheme-ը ցույց է տալիս արձանագրության ձևը, օրինակ՝ http կամ https, host-ը՝ նպատակային համակարգը, path-ը՝ ռեսուրսի ուղին: Query մասը փոխանցում է լրացուցիչ պարամետրեր և սկսվում է հարցական նշանով, իսկ fragment-ը ցույց է տալիս փաստաթղթի ներքին հատվածը և սկսվում է # նշանով: Fragment-ը սովորաբար չի ուղարկվում HTTP հարցման մեջ, որովհետև այն մշակվում է բրաուզերի կողմից:

Օրինակ՝ `https://example.com:443/products/7?lang=hy#reviews` հասցեում `/products/7-` ը path-ն է, `lang=hy`-ը query պարամետր է, իսկ `reviews`-ը՝ էջի հատված: Query-ում մի քանի պարամետրերը բաժանվում են & նշանով: Հատուկ նշաններն ու բացատները URL-ում կոդավորվում են percent-encoding եղանակով: Գաղտնի տվյալներ, օրինակ՝ գաղտնաբառեր, query string-ում տեղադրել պետք չէ, քանի որ URL-ը կարող է պահպանվել պատմության, log-երի և analytics համակարգերի մեջ:

Մաս	Օրինակ	Նշանակություն
scheme	https	Կապի կանոնն ու անվտանգության ձևը
host	example.com	Ծառայության հասցեն
port	443	Ծառայության ցանցային պորտը
path	/products/7	Ռեսուրսի ուղին
query	?lang=hy	Լրացուցիչ պարամետրեր
fragment	#reviews	Էջի ներքին հատված

7. Proxy, Reverse Proxy և CDN

Forward proxy-ն ներկայացնում է հաճախորդներին, reverse proxy-ն՝ սերվերային համակարգը, իսկ CDN-ը բաշխում է բովանդակությունը աշխարհագրորեն տարբեր հանգույցներում: Այս լուծումները կարող են տեխնիկապես նման միջնորդական քայլեր կատարել, բայց դրանց պատասխանատվությունը տարբեր է: Forward proxy-ի կարգավորումը սովորաբար վերահսկում է օգտատիրոջ կազմակերպությունը, reverse proxy-ինը՝ կայքի սեփականատերը: CDN-ը ևս սովորաբար կառավարվում է կայքի սեփականատիրոջ կողմից կամ օգտագործվում է որպես արտաքին ծառայություն:

CDN-ը հատկապես օգտակար է պատկերների, CSS/JavaScript ֆայլերի, տեսանյութերի և այլ ստատիկ բովանդակության համար: Երբ օգտատերը պահանջում է ֆայլ, CDN-ը փորձում է այն վերադարձնել մոտակա edge հանգույցի քննիչ: Եթե պատճենը չկա կամ ժամկետանց է, edge հանգույցը դիմում է origin սերվերին, ստանում պատասխանը և կարող է այն պահել հաջորդ հարցումների

համար: Այս ճանապարհը նվազեցնում է latency-ն և origin-ի ծանրաբեռնվածությունը:

CDN-ը կախարդական արագացուցիչ չէ. քեշավորման սխալ կանոնները կարող են օգտատերերին ցույց տալ հին բովանդակություն կամ նույնիսկ գաղտնի տվյալներ: Անհատականացված պատասխանները սովորաբար չեն պահվում ընդհանուր քեշում առանց հատուկ նախագծման: Թարմացման ժամանակ կարող է անհրաժեշտ լինել cache purge կամ ֆայլերի versioning, օրինակ՝ app.v2.css անունը: Լավ ճարտարապետությունը հստակ որոշում է՝ որ բովանդակությունը որտեղ և որքան ժամանակ կարելի է պահել:

Տեխնոլոգիա	Ում անունից է գործում	Հիմնական նպատակ
Forward Proxy	Հաճախորդի	Ելքի վերահսկում, ֆիլտրում, գաղտնիություն
Reverse Proxy	Սերվերի	Պաշտպանություն, routing, load balancing
CDN	Բովանդակություն տրամադրողի	Մոտ edge-ից արագ առաքում և origin-ի թեթևացում

8. Քեշավորման կառավարում

Քեշը պահպանում է նախկին պատասխանի պատճենը, որպեսզի նույն ռեսուրսը կրկին պահանջելիս ամբողջ աշխատանքը չկրկնվի: Browser cache-ը ծառայում է մեկ օգտատիրոջ սարքին, shared cache-ը կարող է ծառայել շատ օգտատերերի, իսկ reverse proxy կամ CDN cache-ը գտնվում է origin սերվերի դիմաց: Քեշավորումը նվազեցնում է բեռնման ժամանակը, ցանցային տրաֆիկը և սերվերի ծանրաբեռնվածությունը: Փոխարենը պետք է կառավարել տվյալների թարմությունն ու գաղտնիությունը:

Cache-Control վերնագիրը սահմանում է հիմնական կանոնները: max-age=3600-ը նշանակում է, որ պատասխանը կարող է թարմ համարվել 3600 վայրկյան, no-store-ը արգելում է պահպանումը, իսկ private-ը նախատեսում է միայն մասնավոր քեշի օգտագործում: no-cache-ը չի նշանակում «երբեք չպահել». այն պահանջում է պատասխանը օգտագործելուց առաջ վերահաստատել սերվերի հետ: Այս տարբերությունը հաճախ սխալ է ընկալվում:

ETag-ը ներկայացնում է ռեսուրսի կոնկրետ տարբերակի նույնացուցիչը: Հաջորդ հարցման ժամանակ բրաուզերը կարող է այն ուղարկել If-None-Match վերնագրով, և եթե տարբերակը չի փոխվել, սերվերը վերադարձնում է 304 Not Modified առանց ամբողջ մարմնի: Այդ դեպքում բրաուզերը օգտագործում է իր պահած պատճենը: Նման conditional request-ը միավորում է թարմության ստուգումն ու տրաֆիկի խնայողությունը:

```
Cache-Control: public, max-age=3600
ETag: "file-version-42"
If-None-Match: "file-version-42"
HTTP/1.1 304 Not Modified
```

9. HTTP հարցում և պատասխան

HTTP-ը client-server սկզբունքով գործող կիրառական արձանագրություն է: Հաճախորդը նախաձեռնում է հարցումը, իսկ սերվերը վերադարձնում է պատասխան: Հարցումը սովորաբար պարունակում է method, target/path, headers և որոշ մեթոդների դեպքում body: Պատասխանը պարունակում է status code, headers և անհրաժեշտության դեպքում response body:

Վերնագրերը փոխանցում են հիմնական բովանդակությունից դուրս ծառայողական տեղեկություն: Content-Type-ը հայտնում է տվյալների տեսակը, Authorization-ը կարող է փոխանցել մուտքի հավաստում, Accept-ը՝ հաճախորդի նախընտրած պատասխանի ձևաչափը: Content-Length-ը նկարագրում է մարմնի չափը, իսկ Cache-Control-ը՝ պահպանման կանոնները: Վերնագրերի անունները իմաստային առումով case-insensitive են, թեև գործիքները կարող են ցուցադրել տարբեր գրությամբ:

Օրինակ՝ ապրանքների ցանկ ստանալու համար բրաուզերը կարող է ուղարկել GET /api/products հարցում Accept: application/json վերնագրով: Սերվերը կարող է վերադարձնել 200 OK, Content-Type: application/json և JSON զանգված: Եթե հարցման համար պահանջվում է նույնականացում, բայց token չկա, սերվերը կարող է վերադարձնել 401: Developer Tools-ը թույլ է տալիս տեսնել այս բոլոր մասերը առանձին դաշտերով:

```
GET /api/products HTTP/1.1
Host: example.com
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

[{"id": 1, "name": "Book"}]
```

10. HTTP մեթոդներ

HTTP մեթոդը նկարագրում է հաճախորդի ակնկալած գործողության իմաստը: GET-ը նախատեսված է տվյալ ստանալու, POST-ը՝ տվյալ ուղարկելու կամ նոր գործողություն նախաձեռնելու, PUT-ը՝ ռեսուրսը ամբողջությամբ փոխարինելու, PATCH-ը՝ մասնակի փոփոխելու, DELETE-ը՝ հեռացնելու համար: Սերվերը կարող է նույն path-ի համար տարբեր մեթոդներով տարբեր գործողություններ իրականացնել: Օրինակ՝ GET /products-ը կարողում է ցուցակը, իսկ POST /products-ը ստեղծում է նոր տարր:

Մեթոդներն ունեն safe և idempotent հատկություններ: Safe մեթոդը, օրինակ GET-ը, ըստ նշանակության չպետք է փոխի սերվերի վիճակը: Idempotent գործողության բազմակի նույնական կատարումը պետք է ունենա նույն վերջնական ազդեցությունը, ինչ մեկ կատարումը: PUT-ը և DELETE-ը սովորաբար համարվում են idempotent: POST-ը սովորաբար idempotent չէ, որովհետև կրկնվող հարցումը կարող է ստեղծել մի քանի ռեսուրս:

Մեթոդի ընտրությունը միայն անվանում չէ, այլ API-ի պայմանագիր: Եթե տվյալների ջնջումը կատարվում է GET հարցմամբ, բրաուզերի նախաբեռնումը կամ crawler-ը կարող է պատահաբար փոփոխություն առաջացնել: Եթե ամբողջական և մասնակի

թարմացումները տարբերակված չեն, հաճախորդը կարող է ակամա ջնջել չուղարկված դաշտերը: Հետևաբար method-ը պետք է համապատասխանի գործողության իրական իմաստին:

Մեթոդ	Նպատակ	Օրինակ
GET	Կարդալ տվյալ	GET /products/7
POST	Ստեղծել կամ գործողություն կատարել	POST /products
PUT	Ամբողջությամբ փոխարինել	PUT /products/7
PATCH	Մասնակի փոփոխել	PATCH /products/7
DELETE	Հեռացնել ռեսուրսը	DELETE /products/7
HEAD	Ստանալ միայն պատասխանի վերնագրերը	HEAD /image.jpg

11. HTTP կարգավիճակի կոդեր

Status code-ը երեքանիշ թիվ է, որը հայտնում է հարցման արդյունքի ընդհանուր իմաստը: Առաջին թվանշանը ցույց է տալիս խումբը՝ 1xx տեղեկատվական, 2xx հաջող, 3xx վերաուղղման կամ պայմանական պատասխանի, 4xx հաճախորդի հարցման խնդիր, 5xx սերվերի խնդիր: Կոդը չի փոխարինում պատասխանի մանրամասն հաղորդագրությանը, բայց թույլ է տալիս ծրագրին արագ որոշել հետագա քայլը: Front-end-ը պետք է ստուգի ոչ միայն body-ն, այլև status code-ը:

200 OK-ը ընդհանուր հաջող պատասխան է, 201 Created-ը հարմար է նոր ռեսուրս ստեղծելուց հետո, իսկ 204 No Content-ը՝ երբ գործողությունը հաջող է, բայց body պետք չէ: 400 Bad Request-ը ցույց է տալիս սխալ հարցում, 401 Unauthorized-ը՝ նույնականացման անհրաժեշտություն, 403 Forbidden-ը՝ իրավունքի բացակայություն, 404 Not Found-ը՝ ռեսուրսի չգտնվելը: 422 Unprocessable Content-ը հաճախ կիրառվում է վավերացման սխալների համար: 429 Too Many Requests-ը հայտնում է հարցումների հաճախականության սահմանափակման մասին:

5xx կոդերը նշանակում են, որ սերվերային կոդմը չի կարողացել ճիշտ մշակել ընդունելի հարցումը: 500 Internal Server Error-ը ընդհանուր ներքին սխալ է, 502 Bad Gateway-ը հաճախ առաջանում է, երբ proxy-ն upstream-ից անվավեր պատասխան է ստանում, իսկ 503 Service Unavailable-ը՝ երբ ծառայությունը ժամանակավորապես հասանելի չէ: 504 Gateway Timeout-ը ցույց է տալիս, որ gateway-ը կամ proxy-ն ժամանակին պատասխան չի ստացել: Սխալի աղբյուրը գտնելիս պետք է դիտել և՛ status code-ը, և՛ response body-ն, և՛ server log-երը:

Կոդ	Իմաստ	Սովորական իրավիճակ
200	Հաջող հարցում	Տվյալը վերադարձվել է
201	Ռեսուրսը ստեղծվել է	POST-ը ստեղծել է նոր գրառում
304	Քեշի պատճենը դեռ կիրառելի է	ETag-ը չի փոխվել
400	Սխալ հարցում	Անվավեր JSON կամ պարամետր

401	Պետք է նույնականանալ	Token-ը բացակայում է կամ անվավեր է
403	Գործողությունը թույլատրված չէ	Օգտատերը չունի իրավունք
404	Ռեսուրսը չի գտնվել	URL-ը կամ id-ն սխալ է
500	Սերվերի ներքին սխալ	Ծրագրային չմշակված սխալ
502	Gateway-ի սխալ	Proxy-ն upstream-ից ճիշտ պատասխան չի ստացել

12. HTTPS, HTTP 1.1, HTTP 2 և HTTP 3

HTTPS-ը առանձին կիրառական արձանագրություն չէ, այլ HTTP հաղորդակցություն՝ TLS պաշտպանությամբ: TLS-ը կոդավորում է տվյալները, հաստատում է սերվերի ինքնությունը հավաստագրի միջոցով և օգնում է հայտնաբերել փոխանցման ընթացքում փոփոխությունները: Սա պաշտպանում է գաղտնաբառերը, session cookie-ները և այլ տվյալներ ցանցում գաղտնալսումից: HTTPS-ը չի երաշխավորում, որ կայքի բովանդակությունը վստահելի է. այն պաշտպանում է կապը ընտրված սերվերի հետ:

HTTP/1.1-ում բրաուզերը հաճախ օգտագործում է մի քանի TCP կապ՝ զուգահեռ ռեսուրսներ ստանալու համար: HTTP/2-ը մեկ կապի ներսում ստեղծում է բազմաթիվ հոսքեր, օգտագործում է երկուական framing և սեղմում է header-ները: Դրա շնորհիվ մի քանի հարցումներ կարող են միաժամանակ աշխատել նույն կապով: Այնուամենայնիվ TCP փաթեթի կորուստը կարող է ժամանակավորապես կանգնեցնել նույն կապի հոսքերը:

HTTP/3-ը HTTP իմաստաբանությունը փոխանցում է QUIC-ի վրա, որը սովորաբար աշխատում է UDP-ի միջոցով և ունի ներկառուցված TLS: QUIC-ի անկախ հոսքերը նվազեցնում են այն դեպքերը, երբ մեկ հոսքի կորուստը կանգնեցնում է մյուս հոսքերը: HTTP/3-ը կարող է արագացնել կապի հաստատումը հատկապես փոփոխական կամ հեռավոր ցանցերում: Բովանդակության իմաստը՝ մեթոդները, status code-երը և headers-ը, հիմնականում նույնն են մնում բոլոր տարբերակներում:

Տարբերակ	Փոխադրման հիմք	Գլխավոր առանձնահատկություն
HTTP/1.1	TCP	Պարզ տեքստային ձև և կապերի սահմանափակ զուգահեռություն
HTTP/2	TCP	Multiplexing և header compression մեկ կապում
HTTP/3	QUIC՝ UDP-ի վրա	Անկախ հոսքեր և արագ կապի հաստատում

13. Այլ հիմնական արձանագրություններ

Վեբ հաղորդակցությունը կազմված է մի քանի փոխկապակցված արձանագրություններից: TCP-ը ապահովում է byte stream-ի հերթական և հոսալի փոխանցում, իսկ QUIC-ը նման հնարավորություններ իրականացնում է UDP-ի վրա՝ հավելյալ հոսքերի և անվտանգության մեխանիզմներով: TLS-ը պաշտպանում է

կապը, իսկ HTTP-ը սահմանում է հարցման և պատասխանի իմաստը: Այս շերտերը միասին են ստեղծում օգտատիրոջ տեսած վեբ փոխազդեցությունը:

WebSocket-ը ստեղծում է երկարատև երկկողմ կապ, որի ընթացքում և՛ հաճախորդը, և՛ սերվերը կարող են հաղորդագրություն ուղարկել առանց յուրաքանչյուր անգամ նոր HTTP հարցում սկսելու: Այն հարմար է chat-ի, առցանց խաղերի, live dashboard-ների և իրական ժամանակում ծանուցումների համար: Սկզբնական կապը սովորաբար սկսվում է HTTP handshake-ով, հետո անցնում WebSocket հաղորդակցությանը: Եթե տվյալները հազվադեպ են թարմացվում, սովորական HTTP polling-ը երբեմն ավելի պարզ լուծում է:

SSH-ը անվտանգ հեռավար կառավարման արձանագրություն է: Այն օգտագործվում է սերվեր մուտք գործելու, հրամաններ կատարելու, ֆայլեր փոխանցելու և Git repository-ների հետ աշխատելու համար: SSH բանալիները հնարավորություն են տալիս նույնականանալ առանց հաշվի գաղտնաբառը յուրաքանչյուր անգամ փոխանցելու: Բանալիի private մասը չպետք է հրապարակվի կամ ուղարկվի repository:

14. VS Code և Նախագծի կառուցվածք

Visual Studio Code-ը կոդի խմբագիր է, որը մեկ պատուհանում միավորում է ֆայլերի Explorer-ը, որոնումը, terminal-ը, debugger-ը և source control-ը: Workspace-ը VS Code-ում բացված նախագիծն է կամ նախագծերի հավաքածուն: Ճիշտ workspace բացելը կարևոր է, որովհետև terminal-ը, relative path-երը և Git-ը աշխատում են այդ պանակի համատեքստում: Եթե բացված է սխալ պանակ, հրամանները կարող են կատարվել սխալ տեղում:

Պարզ ստատիկ նախագծում index.html-ը սովորաբար հիմնական էջն է, css պանակում պահվում են ոճերը, js պանակում JavaScript ֆայլերը, assets կամ images պանակում մեդիա կյուլթերը: Ֆայլերի անվանումները ցանկալի է գրել փոքրատառ, առանց բացատների և հասկանալի բառերով: Relative path-ը հաշվարկվում է ընթացիկ ֆայլի դիրքից, ուստի կառուցվածքի փոփոխությունը կարող է կոտրել հղումները: Նախագծի կառուցվածքը պետք է լինի պարզ այնքան, որքան թույլ է տալիս աշխատանքի ծավալը:

README.md-ը բացատրում է նախագծի նպատակը, տեղադրման կամ գործարկման քայլերը և կարևոր պայմանները: .gitignore-ը նշում է ֆայլերն ու պանակները, որոնք չպետք է մտնեն Git պատմության մեջ, օրինակ՝ գաղտնի .env ֆայլը կամ ավտոմատ ստեղծվող dependency պանակները: Այնուամենայնիվ .gitignore-ը չի հեռացնում արդեն commit արված գաղտնիքը պատմությունից: Գաղտնի տվյալը պատահաբար commit անելու դեպքում այն պետք է նաև անմիջապես փոխել կամ չեղարկել:

```
my-web-project/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   └── app.js  
├── assets/  
└── README.md
```

15. Developer Tools

Բրաուզերի Developer Tools-ը վեր էջի կառուցվածքն ու աշխատանքը ուսումնասիրելու գործիքների հավաքածու է: Elements բաժինը ցույց է տալիս բրաուզերի կառուցած DOM-ը և կիրառված CSS կանոնները: Console-ը ցուցադրում է JavaScript սխալները, ծրագրի հաղորդագրությունները և թույլ է տալիս փոքր հրամաններ կատարել: Sources բաժնում կարելի է դիտել բեռնված ֆայլերը և debugger-ով քայլ առ քայլ ուսումնասիրել կոդը:

Network բաժինը ցույց է տալիս էջի բոլոր ցանցային հարցումները: Յուրաքանչյուր գրառման մեջ կարելի է տեսնել անունը կամ URL-ը, մեթոդը, status code-ը, initiator-ը, չափը և տևողությունը: Հարցումը բացելիս Headers բաժինը ցույց է տալիս request և response headers-ը, Payload-ը՝ ուղարկված տվյալները, Response կամ Preview-ը՝ ստացված բովանդակությունը: Timing բաժինը օգնում է հասկանալ, թե ժամանակը ծախսվել է կապի, սպասման, թե ներբեռնման վրա:

Network-ը ճիշտ ուսումնասիրելու համար նախ բացել Developer Tools-ը, հետո միացնել Preserve log-ը միայն անհրաժեշտության դեպքում և վերաբեռնել էջը: Fetch/XHR ֆիլտրը օգնում է առանձնացնել API հարցումները, իսկ Disable cache-ը՝ թեստի ժամանակ բացառել browser cache-ի ազդեցությունը: Կարմիր հարցումը միշտ պետք է բացել և ուսումնասիրել status code-ը ու response body-ն: Միայն Console-ի ընդհանուր սխալը հաճախ բավարար չէ իրական պատճառը հասկանալու համար:

1. Բացել էջը և սեղմել F12 կամ ընտրել Inspect:
2. Բացել Network բաժինը և վերաբեռնել էջը:
3. Ընտրել որևէ document, stylesheet կամ fetch հարցում:
4. Գտնել Request URL-ը, մեթոդը և Status Code-ը:
5. Բացել Response Headers-ը և գտնել Content-Type ու Cache-Control դաշտերը:
6. Timing բաժնում համեմատել սպասման և ներբեռնման ժամանակները:

16. Git, GitHub և դասի ամփոփում

Git-ը տեղային version control համակարգ է, որը պահպանում է նախագծի փոփոխությունների պատմությունը: GitHub-ը առցանց հարթակ է Git repository-ների պահպանման, համագործակցության և կոդի դիտարկման համար: Git-ը կարող է աշխատել առանց GitHub-ի, իսկ GitHub-ում տեղադրված նախագիծը սովորաբար ունի Git պատմություն: Այս տարբերությունը կարևոր է, որովհետև commit-ը տեղային գործողություն է, իսկ push-ը փոփոխությունները փոխանցում է remote repository:

Սովորական աշխատանքային հերթականությունն է՝ փոփոխել ֆայլը, ստուգել git status-ը, ընտրել փոփոխությունը git add-ով և ստեղծել commit: Commit-ը պետք է նկարագրի մեկ ամբողջական ու հասկանալի փոփոխություն: Դրանից հետո git push հրամանով commit-ները ուղարկվում են GitHub: Եթե մի քանի մարդ աշխատում է նույն նախագծի վրա, նախ կարող է անհրաժեշտ լինել ստանալ մյուսների փոփոխությունները և լուծել հնարավոր conflict-ները:

Այս դասի թեմաները կազմում են մեկ ամբողջական շղթա: Բրաուզերը հարցում է ստեղծում, արձանագրությունները այն փոխանցում են, proxy/CDN միջնորդները

կարող են վերահսկել կամ արագացնել ճանապարհը, սերվերը վերադարձնում է status code և բովանդակություն, իսկ Developer Tools-ը հնարավորություն է տալիս դիտել արդյունքը: VS Code-ում մշակողը փոխում է նախագծի ֆայլերը, իսկ Git-ը պահպանում է այդ փոփոխությունների պատմությունը: Ուսանողը պետք է կարողանա ոչ միայն սահմանել հասկացությունները, այլև ցույց տալ դրանք գործնական գործիքներում:

```
git status
git add index.html
git commit -m "Add first web page"
git push
```

-